

IA-corus-intranet

CorusIntranetEngine v2.0

Resumen del proyecto y stack tecnológico

Asistente de IA corporativo interno para la empresa Corus. Chatbot basado en RAG (Retrieval-Augmented Generation) que responde preguntas en lenguaje natural sobre los procedimientos internos de Parafiscales y Pensiones, apoyándose exclusivamente en los manuales oficiales de la empresa.

Repositorio: [sebassisamiami-sketch/IA-corus-intranet](https://github.com/sebassisamiami-sketch/IA-corus-intranet)

Documento generado automáticamente

1. Descripción del proyecto

CorusIntranetEngine v2.0 es un asistente de IA corporativo interno (una intranet inteligente) para la empresa Corus. Es un chatbot que responde preguntas en lenguaje natural sobre los procedimientos internos de Parafiscales y Pensiones, basándose exclusivamente en los manuales oficiales de la empresa (archivos PDF).

Utiliza la técnica RAG (Retrieval-Augmented Generation): en lugar de "inventar" respuestas, la IA busca el documento correcto entre los manuales y genera la respuesta únicamente con esa información, mostrando además las fuentes para que puedan verificarse.

El dominio de negocio gira en torno a herramientas como webMethods (Software AG / BPM), Service Manager (gestión de radicados / tickets) y DBeaver (consultas SQL).

2. Tecnologías (stack técnico)

Lenguaje	Python 3.11
Frontend / App web	Streamlit
Framework de IA	LangChain
LLM (modelo de lenguaje)	OpenAI GPT-3.5-turbo (chat), GPT-4o / GPT-4o-mini (visión)
Embeddings	OpenAI (text-embedding-ada-002)
Base de datos vectorial	ChromaDB
Transcripción audio/video	OpenAI Whisper (whisper-1)
Procesamiento de PDF	pypdf
Generación de PDF	fpdf2
Generación de PowerPoint	python-pptx
Datos	pandas, tiktoken
Seguridad	bcrypt (hash de contraseñas), tokens HMAC-SHA256 firmados
Audio/Video (sistema)	ffmpeg
Grabación de micrófono	streamlit-mic-recorder
Contenedores	Docker (Python 3.11-slim)
Hosting	Streamlit Community Cloud y/o Render (con Docker)
Repositorio / CI	GitHub (auto-deploy en main)

3. Arquitectura y módulos principales

El flujo general es: Usuario (navegador) -> App Streamlit -> Motor IA (RAG) -> OpenAI. Los PDFs se almacenan en GitHub y se indexan como vectores en ChromaDB.

Archivos clave

- **app_corus.py:** Aplicación principal Streamlit. Login, chat estilo ChatGPT/Gemini, panel de administración, seguridad y control del servidor. Es el archivo central.
- **ia_motor.py:** Motor de IA (patrón Singleton). Implementa el RAG: búsqueda vectorial, selección inteligente de un único documento, continuidad de conversación y modo experto.
- **procesar_datos.py:** Lee los PDFs, los divide en fragmentos (chunks) y crea/actualiza la base vectorial ChromaDB.
- **chat_procesos.py:** Gestiona la conversación por usuario: memoria, historial persistente y filtros de saludos/cierres para ahorrar tokens.
- **vision_chat.py:** Análisis de imágenes (capturas de BPM / Service Manager) usando modelos de visión GPT-4o.
- **video_a_manual.py:** Convierte un video/audio en un manual PDF: extrae audio con ffmpeg, transcribe con Whisper, resume con GPT y genera el PDF.
- **invite_links.py:** Genera enlaces de invitación temporales (tokens firmados con HMAC) y tokens de sesión para acceso sin login.
- **copiloto_reunion.py:** App separada: copiloto de reuniones que graba el micrófono, transcribe y responde preguntas del proyecto en vivo.
- **control_servidor.py:** Script CLI para poner la app en modo mantenimiento remotamente vía la API de GitHub.
- **generar_presentacion.py:** Genera una presentación PowerPoint profesional del proyecto.
- **consultar_datos.py:** Herramienta de depuración (comentada, no se usa en producción).

4. Funcionalidades destacadas

- **Chat con IA (RAG):** Responde sobre procesos con fuentes verificables, sin inventar.
- **Roles y acceso:** Administrador, Analista e Invitado (por enlace), cada uno con permisos y vistas distintas.
- **Panel de administración:** Gestión de usuarios, procesamiento de PDFs, estado de la BD, registros de acceso, estadísticas de IA y control del servidor.
- **Auto-indexado dinámico:** Detecta e indexa sola cualquier carpeta nueva de PDFs subida a GitHub al arrancar (necesario porque Streamlit Cloud borra el disco en cada redesplicue).
- **Análisis de imágenes:** Sube una captura de pantalla y la IA ayuda con el caso (visión).
- **Video a Manual:** Convierte grabaciones en manuales PDF automáticamente.
- **Modo experto:** Alterna entre responder solo con los manuales o con conocimiento general del dominio (BPM, webMethods, etc.).
- **Seguridad robusta:** bcrypt, bloqueo tras 5 intentos fallidos, auto-logout por inactividad (15 min), enlaces de invitación firmados y caducables, modo mantenimiento.
- **Personalización visual:** CSS elaborado que imita la interfaz de ChatGPT/Gemini y oculta la marca de Streamlit.

5. Observaciones

- **Versiones inconsistentes:** requirements.txt usa Streamlit 1.28 y LangChain 0.0.354, mientras QUICK_START.md menciona Streamlit 1.38 y LangChain 0.2.16. El archivo real de dependencias usa las versiones antiguas.
- **Función duplicada:** En video_a_manual.py hay una función resumir_a_manual duplicada; la segunda definición sobrescribe a la primera (probable descuido).
- **Idioma:** El proyecto está orientado a hispanohablantes: código, comentarios y UI están en español.